Proyecto 2 - Anexo de Uso de IA

Uso de IA en la Consola:

Para que el codigo quedara bien organizado y estructura se le pidió a Claude un pseudocódigo base de la consola diciéndole como quería que quedara y mandandole una idea de código para que supiera que se pedía. Con esto se escribió el código de la consola y finalmente se volvió a mandar a la IA para corregir errores y entender el porque de algunos errores que salían.

Ejemplo

Necesito una consola con un main distinto por tipo de usuario : administrador, empleado, entrenador y socio, todo en el mismo proyecto Eclipse. Mi idea es una clase base abstracta con el login y el ciclo del menú, y una clase por cada opción del menú:

```
public interface OpcionMenu {
    String getTitulo();
    void ejecutar() throws ReglaNegocioException;
}

public abstract class ConsolaBase {
    protected final ClubSeneca club;
    protected final LectorConsola lector;
    protected abstract boolean aceptaUsuario(Usuario usuario);
    protected abstract String getNombreConsola();
    protected abstract List<OpcionMenu> crearOpciones(Usuario
usuario);
}

public class ConsolaSocio extends ConsolaBase {
    public static void main(String[] args) {
        ClubSeneca club = cargarClub();
        new ConsolaSocio(club, new LectorConsola()).iniciar();
    }
}
```

Dame un pseudocódigo de iniciar, autenticar, y ejecutarMenu en ConsolaBase. Quiero que no queden clases demasiado grandes y que agregar una opción nueva sea solo crear otra clase. No escribas la lógica del negocio.

Uso de IA en los Test:

Con el fin de crear los test en JUnit para el código del Proyecto 1, se le solicitó a Claude que realizará un paquete java con un esqueleto para todos los metodos de los test realizados, con ToDo para realizar más cómodamente y mejor organizado la implementación de los test.

EJ:

```
@Test
void testClubNuevo()
{
    // TODO
}

@Test
void testCrearSocio()
{
    // TODO
}

@Test
void testCrearSocioConLoginRepetido()
{
    // TODO
}

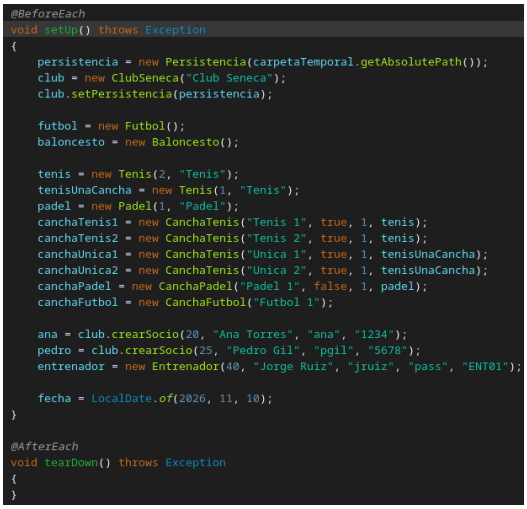
@Test
void testCrearSocioGuardaEnDisco() throws Exception
{
    // TODO
}

@Test
void testIniciarSesion()
{
    // TODO
}

@Test
void testIniciarSesionConDatosIncorrectos()
{
    // TODO
}

@Test
void testCrearEquipo()
{
    // TODO
}
```

También se le pidió organizar los datos usados en los test en cada clase en el @BeforeEach y @AfterEach, junto con los mensajes de error de los assert, para facilitar la implementación y agilizar la construcción eficiente de los test evitando pasos repetitivos.
EJ:



```
@BeforeEach
void setUp() throws Exception
{
    persistencia = new Persistencia(carpetaTemporal.getAbsolutePath());
    club = new ClubSeneca("Club Seneca");
    club.setPersistencia(persistencia);

    futbol = new Futbol();
    baloncesto = new Baloncesto();

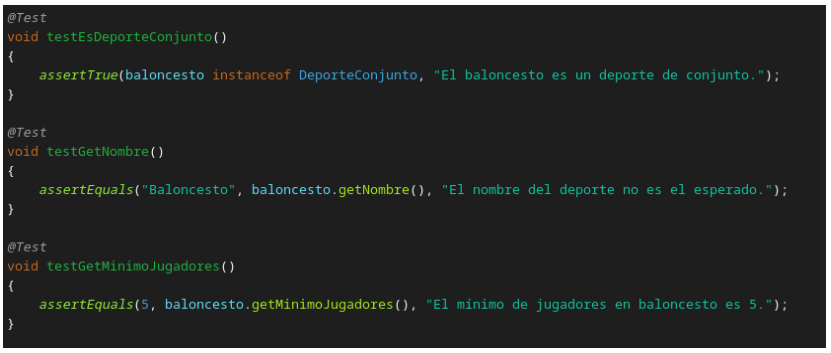
    tenis = new Tenis(2, "Tenis");
    tenisUnaCancha = new Tenis(1, "Tenis");
    padel = new Padel(1, "Padel");
    canchaTenis1 = new CanchaTenis("Tenis 1", true, 1, tenis);
    canchaTenis2 = new CanchaTenis("Tenis 2", true, 1, tenis);
    canchaUnica1 = new CanchaTenis("Unica 1", true, 1, tenisUnaCancha);
    canchaUnica2 = new CanchaTenis("Unica 2", true, 1, tenisUnaCancha);
    canchaPadel = new CanchaPadel("Padel 1", false, 1, padel);
    canchaFutbol = new CanchaFutbol("Futbol 1");

    ana = club.crearSocio(20, "Ana Torres", "ana", "1234");
    pedro = club.crearSocio(25, "Pedro Gil", "pgil", "5678");
    entrenador = new Entrenador(40, "Jorge Ruiz", "jruiz", "pass", "ENT01");

    fecha = LocalDate.of(2026, 11, 10);
}

@AfterEach
void tearDown() throws Exception
{
}
```

Finalmente, aunque el enunciado menciona que no es necesario implementar los test de los getters y setters del proyecto o de los assert de instancias, con el fin de alcanzar un cubrimiento casi total del proyecto, se solicitó la ayuda a la IA para implementar estos métodos de testeo evitando realizar tareas repetitivas.
EJ:



```
@Test
void testEsDeporteConjunto()
{
    assertTrue(baloncesto instanceof DeporteConjunto, "El baloncesto es un deporte de conjunto.");
}

@Test
void testGetNombre()
{
    assertEquals("Baloncesto", baloncesto.getNombre(), "El nombre del deporte no es el esperado.");
}

@Test
void testGetMinimoJugadores()
{
    assertEquals(5, baloncesto.getMinimoJugadores(), "El mínimo de jugadores en baloncesto es 5.");
}
```

Como extra se le pidió que pusiera el código y lo organizara para tener un código mejor realizado.

Uso de IA para estructurar el proyecto

Se usó la IA pensando en estructurar el proyecto apoyándonos de esta para elegir las historias de usuario que se acoplan mejor al sistema que teníamos previamente diseñado y estructurado. Adicionalmente que nos recomendara ideas y nos pudiera dar una idea de como realizar la implementación del código respecto a las historias de usuario tanto de las del banco como una guía para la creación de las nuevas historias de usuario. Todo esto con el fin de realizar la entrega de manera completa pero siendo hecha por nosotros, la IA fue usada con el fin de poder alcanzar un trabajo completo y de servirnos como apoyo para poder empezar con el proyecto.

Claude tenemos nuevo proyecto y es para el domingo a media noche pero el sábado y domingo andamos ocupados así que empecemos ya. Te mandaré lo de este proyecto y me ayudarás a elegir las historias de usuario que veas que mejor se acoplen a lo que ya llevamos y a darme consejos para construir las nuevas historias de usuario, y los pasos para hacerlo de manera adecuada para sacarnos 5 en esta entrega. De paso dame una guía para realizar el código, no me lo des pero dame la idea de como tiene que realizarse. Adicionalmente que es lo que tenemos que hacer y los entregables que tienen que haber presentes para que no nos bajen puntos

Lo más importante:

- **Historias del banco:** te recomiendo HU-SOC-02, HU-ENT-02 y HU-ADM-02, más HU-SOC-01 de respaldo. Las cuatro ya están casi hechas en su código. Lo que les falta a todas es lo mismo: que el sistema diga por qué rechaza algo. Hoy devuelven `false` o `null`, y deberían lanzar `ReglaNegocioException` con un mensaje.
- **Historias de torneos:** te dejé 12 sugeridas y una de ejemplo completa para usar como plantilla. Lo que más se olvida es poner las entradas y salidas en cada historia.
- **Revisa esto primero:** casi todas las operaciones de `ClubSeneca` guardan en `datos/club.json`. Si corren las pruebas JUnit así, les pueden dañar los datos. En el documento explico cómo evitarlo.
- **Uso de IA:** el enunciado no deja generar con IA la lógica de los torneos, por eso solo te di las ideas. Esta conversación tiene que ir en el anexo de uso de IA.